Mid Term Assignment Theory Report

MCA IIIrd

Submitted By:

Name	Roll Number
Ravi Asopa	243301040
Nikhil Singh Bhati	243301031
Jaivardhan Singh Chauhan	243301012

GitHub Link: https://github.com/Xema7/Data-Science-Assignment.git

Introduction

The project is, well, a collection of work done on data. Customer data, to be clear. We used Python, pandas, and some others like seaborn, which are useful. The dataset contained many fields. It had some missing ones. We wanted to process it. Because data cleaning is important before anything else can happen.

The idea was not big but necessary. We learned that cleaning is not small work. It takes time and repetition. The same steps again, sometimes again.

Methodology

We imported the required libraries. NumPy, pandas, matplotlib, seaborn all the good ones. Then, we loaded the dataset named “customers data.csv”. We started checking for basic details, like shape, info, describe.

Then we saw missing values. So many of them. We handled them by filling using mean, median, and mode, but not all together. Only where it felt right.

Duplicates were another problem. Some customers appeared again. But they shouldn’t. So we dropped them by Customer_ID. We also dropped unnecessary features, such as ‘Income_to_Spending_Ratio’ and ‘Age_Group’. Not because we hated them, but because they didn’t help.

Data Cleaning

The cleaning process was calm and repetitive. Checking null values again and again, though it was same result. Still, we checked. Then, we filled missing entries and confirmed again.

Finally, we standardized the data, using StandardScaler, which sounded impressive. Encoding was done too LabelEncoder for categorical columns. The dataset felt lighter, maybe cleaner, but still large.

Sometimes the process broke, sometimes not. We fixed it by re-running. It worked eventually, without major reason why.

Visualization

We plotted heatmaps, distributions they looked colorful. But some were confusing, or maybe too clear. Seaborn helped but sometimes crashed, though not always.

Heatmap showed correlations that were not too strong, or maybe too weak. It was fine either way.

Results

The data became more consistent after preprocessing. Missing values disappeared, duplicates vanished. A few columns were gone too, which was necessary.

We could now analyze customers by their income, satisfaction rating, or purchase frequency. It gave an idea of how different people spend differently, but the same pattern sometimes appears again.

Conclusion

The project taught us cleaning is not just cleaning. It's patience. It's debugging too. We saw null values, filled them, dropped others, and ended up with a dataset ready for further modeling.

Though sometimes, the same steps repeated. It was fine. Because repetition makes it stable, or maybe not, but we accepted that.

Future Work

Maybe later we'll build a model on this data. Classification, clustering, or prediction something more intelligent. But for now, preprocessing is done. And that's good.

Appendix

- Notebook used: Ravi Asopa_Mid Term Assignment.ipynb
- Dataset: customers data.csv

Output ScreenShots

```python
df.shape  # Show rows*columns of dataframe
```

```
(1405, 25)
```

```python
df.info()  # Shows null values and data type of dataframe features
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1405 entries, 0 to 1404
Data columns (total 25 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Customer_ID                1405 non-null   int64
 1   Age                        1294 non-null   float64
 2   Gender                     1301 non-null   object
 3   Annual_Income              1291 non-null   float64
 4   Spending_Score             1291 non-null   float64
 5   City                       1309 non-null   object
 6   Review_Text                1292 non-null   object
 7   Membership_Duration        1306 non-null   float64
 8   Purchase_Frequency         1301 non-null   float64
 9   Satisfaction_Rating        1300 non-null   float64
 10  Income_to_Spending_Ratio   1292 non-null   float64
 11  Age_Group                  1299 non-null   object
 12  Is_High_Spender            1295 non-null   float64
 13  Income_Category            1288 non-null   object
 14  Purchase_Intensity         1297 non-null   float64
 15  Engagement_Score           1282 non-null   float64
 16  Customer_Segment           1283 non-null   object
 17  City_Tier                  1309 non-null   object
 18  Review_Length              1300 non-null   float64
 19  Positive_Feedback          1289 non-null   float64
 20  Avg_Spend_per_Visit        1288 non-null   float64
 21  Loyalty_Index              1294 non-null   float64
 22  Has_Long_Membership        1291 non-null   float64
 23  Digital_Engagement         1300 non-null   float64
 24  Churn(target)              1405 non-null   int64
dtypes: float64(16), int64(2), object(7)
memory usage: 274.5+ KB
```

```python
df.isnull().sum()  # Checking for null values
```

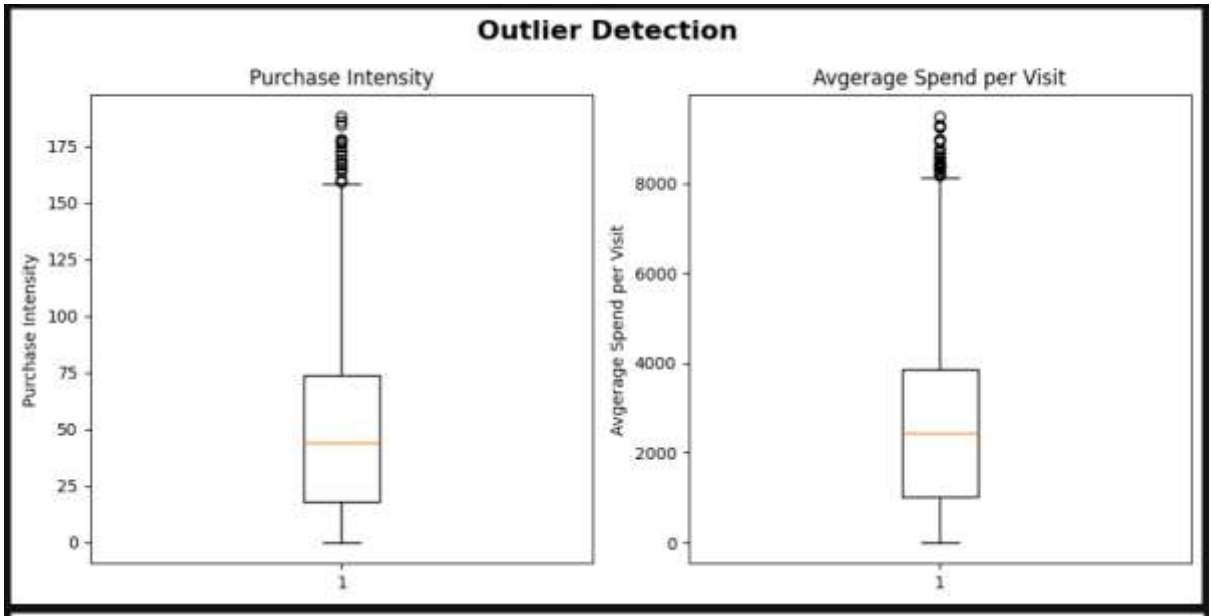
```
Customer_ID                  0
Age                        111
Gender                     104
Annual_Income              114
Spending_Score             114
City                        96
Review_Text                113
Membership_Duration         99
Purchase_Frequency         104
Satisfaction_Rating        105
Income_to_Spending_Ratio   113
Age_Group                  106
Is_High_Spender            110
Income_Category            117
Purchase_Intensity         108
Engagement_Score           123
Customer_Segment           122
City_Tier                   96
Review_Length              105
Positive_Feedback          116
Avg_Spend_per_Visit        117
Loyalty_Index              111
Has_Long_Membership        114
Digital_Engagement         105
Churn(target)                0
dtype: int64
```

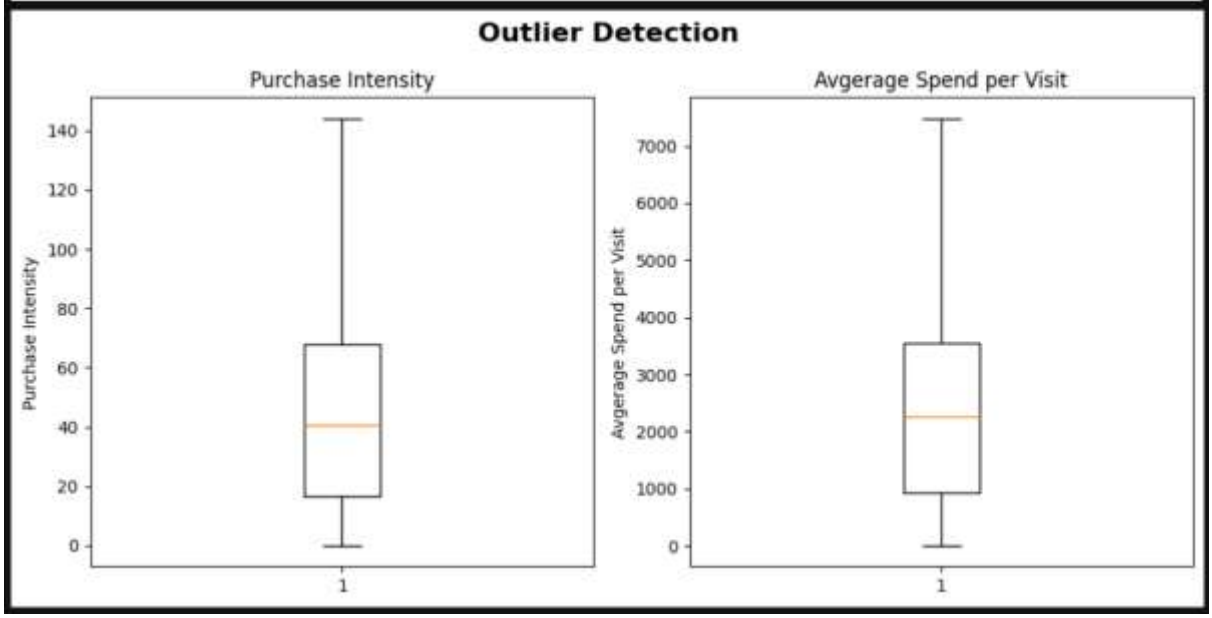
```python
df = df.drop_duplicates(subset=['Customer_ID'])  # Drop duplicate data
```

```python
# Dropping unnecessary features

df.drop(['Customer_ID', 'Income_to_Spending_Ratio', 'Age_Group',
         'Is_High_Spender', 'Income_Category', 'Review_Text',
         'Review_Length', 'Positive_Feedback', 'City_Tier'],
        axis=1, inplace=True)
```

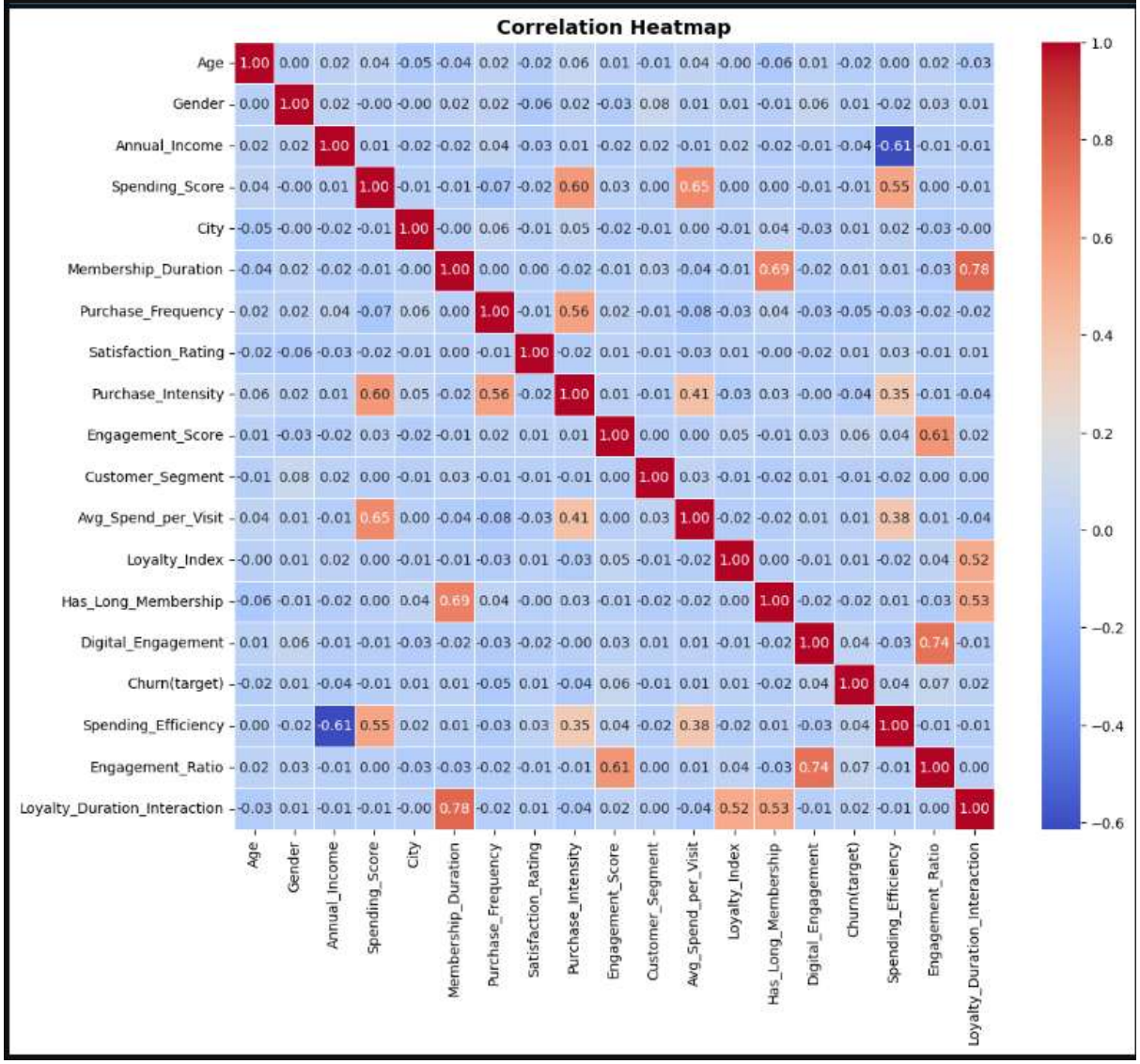
```python
# Filling null values with mean / mode / median

# Selecting features with null values
cols_to_fill = {
    'Age': 'mean',
    'Annual_Income': 'median',
    'Spending_Score': 'mean',
    'Purchase_Frequency': 'mean',
    'Satisfaction_Rating': 'mode',
    'Purchase_Intensity': 'mean',
    'Engagement_Score': 'mean',
    'Customer_Segment': 'mode',
    'Avg_Spend_per_Visit': 'mean',
    'Loyalty_Index': 'mean',
    'Has_Long_Membership': 'mode',
    'Digital_Engagement': 'mean'
}

# Loop to fill null values of selected features
for col, method in cols_to_fill.items():
    if col not in df.columns:
        continue
    if method == 'mean':
        df[col].fillna(df[col].mean(), inplace=True)
    elif method == 'median':
        df[col].fillna(df[col].median(), inplace=True)
    elif method == 'mode':
        df[col].fillna(df[col].mode()[0], inplace=True)
```

```python
# Filling null values with mean / mode / median

# Selecting features with null values
cols_to_fill = {
    'Age': 'mean',
    'Annual_Income': 'median',
    'Spending_Score': 'mean',
    'Purchase_Frequency': 'mean',
    'Satisfaction_Rating': 'mode',
    'Purchase_Intensity': 'mean',
    'Engagement_Score': 'mean',
    'Customer_Segment': 'mode',
    'Avg_Spend_per_Visit': 'mean',
    'Loyalty_Index': 'mean',
    'Has_Long_Membership': 'mode',
    'Digital_Engagement': 'mean'
}

# Loop to fill null values of selected features
for col, method in cols_to_fill.items():
    if col not in df.columns:
        continue
    if method == 'mean':
        df[col].fillna(df[col].mean(), inplace=True)
    elif method == 'median':
        df[col].fillna(df[col].median(), inplace=True)
    elif method == 'mode':
        df[col].fillna(df[col].mode()[0], inplace=True)
```


Outlier Detection
Purchase Intensity
Purchase Intensity
0
25
50
75
100
125
150
175
1
Avgerage Spend per Visit
Avgerage Spend per Visit
0
2000
4000
6000
8000
1


Outlier Detection
Purchase Intensity
Purchase Intensity
0
20
40
60
80
100
120
140
1
Avgerage Spend per Visit
Avgerage Spend per Visit
0
1000
2000
3000
4000
5000
6000
7000
1

```python
# Feature Engineering - creating new features using current features
df['Spending_Efficiency'] = df['Spending_Score'] / df['Annual_Income']
df['Engagement_Ratio'] = df['Digital_Engagement'] * df['Engagement_Score']
df['Loyalty_Duration_Interaction'] = df['Membership_Duration'] * df['Loyalty_Index']
```

```python
# Label encoding categorical features
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
df['City'] = le.fit_transform(df['City'])
df['Customer_Segment'] = le.fit_transform(df['Customer_Segment'])
```

```python
# Standardize data
scaler = StandardScaler()

cols_to_scale = ['Age', 'Annual_Income','Spending_Score', 'Membership_Duration', 'Purchase_Frequency',
                 'Satisfaction_Rating', 'Purchase_Intensity', 'Engagement_Score', 'Avg_Spend_per_Visit', 'Loyalty_Index',
                 'Digital_Engagement', 'Spending_Efficiency', 'Engagement_Ratio', 'Loyalty_Duration_Interaction']

df[cols_to_scale] = scaler.fit_transform(df[cols_to_scale])
```